实验指导书：基于中间语言与指令模板的多语言联合 DFA 构建及鲲鹏环境验证

一、实验目的

本实验旨在通过理论与实践结合，掌握基于 σ-DFA 的后端代码生成流程。具体目标包括：

- 理论构建：根据设计的中间语言（QTAC）和指令模板（Q2ARM），构建 σ-DFA 用于指导代码生成。
- 程序实现：编写程序实现上述 DFA 逻辑，将中间代码转换为特定架构的汇编代码。
- 环境验证：利用 Fact 程序（或类似的测试驱动/验证工具）生成测试用例，得出 ARM 汇编代码。
- 硬件运行：在华为鲲鹏（ARM 架构）环境下完成汇编、链接并运行，验证代码生成的正确性。

二、实验环境

- 硬件平台：华为鲲鹏服务器或开发板（ARM64 架构，如 Kunpeng 920）。
- 操作系统：openEuler、Ubuntu ARM 版或 CentOS ARM 版。
- 开发工具 devkit：
  - 编译器：GCC (aarch64-linux-gnu-gcc) 或 LLVM/Clang。
  - 汇编器/链接器：AS, LD。
  - 调试工具：GDB。
- 实验素材：
  - 自定义的中间语言（QTAC）定义文件。
  - 指令模板库（Q2ARM）。

- Fact 测试程序源码。

三、实验原理

- 语言簇 $\mathcal{L}$
  - 根据 QTAC 和 Q2ARM 设计词法记号，使得中间代码（QTAC 程序）的词法单元都与 Q2ARM 中模式匹配，也就是可转换 ARM 指令段的 QTAC 指令段。
- 多语言联合 DFA：
  - 对于正则语言簇 $\mathcal{L}$，如果元素两两不相交，那么就有 σ-DFA($\mathcal{L}$)。额外的附加信息是需要的，以便于识别为词法记号后进行正确转换。
- 指令模板匹配：
  - 对于词法记号串逐一转换为对应的 ARM 指令（可能多条），形成 ARM 程序作为结果。记号内的附加信息用于生成完整 ARM 指令。在记号之间需考虑正确使用寄存器。

四、实验步骤

步骤 1：设计正则语言簇 $\mathcal{L}$

- 分析、梳理 QTAC 和 Q2ARM：
  - 设 Σ 为输入符号集合（IR 操作码+操作数类型）。
  - 对于模板表下边的注解可用附加信息的办法解决
- 设计 $\mathcal{L}$：
  - 对照 Q2ARM 中 QTAC 指令段模板，构建一个正则语言，如果这些正则语言都两两不相交，就将它们的集

合作为 $\mathcal{L}$，否则通过给模板附加注释达到构成 $\mathcal{L}$ 的条件上述。

步骤 2：构建 σ-DFA（设计与建模）

- 定义状态与转移：
  - 设 Q 为状态集合， Σ 为输入符号集合（已有）。
  - 设计状态转移函数 $\delta: Q \times \Sigma \to Q$ 。
  - 例如：当当前状态为初始状态，当前输入符号为 IF 时，状态转移至 S_IF。再读入符号 LABEL 时，转移去的状态 S_IF_LABEL（前最大化原则）。
  - 考虑到识别和存储词法单元的附加信息（或称为模板参数）。
- 构建 σ-DFA($\mathcal{L}$)的转移表 DELTA[][]：
  - 确定 DELTA 的行列数量为$|Q|$和$|\Sigma|$，确定初始状态，确定接受状态及其标签。
  - 确定 DELTA 中的附加信息管理方式：
    - 可另建表保存附加信息，附加信息可能用于状态转换，也可用于生成 ARM 指令。
  - 注意避免下面紧耦合的过程：
    - 初始态 -> 读取`LOAD` -> 检查寻址模式 -> 输出 `LDR` (ARM)。
    - 建议采用两遍处理，第一遍输出记号串，第二遍（当作词法分析框架中的后处理）将词法记号对照 ARM 指令模式逐一转换输出。

- （这一步不做要求）在后处理中可考虑虚拟寄存器与 q 在指令间的对应。

步骤 3：程序实现代码生成器

- 有硬编码、表驱动、自动生成这三种典型实现方式。任选一种。
- 实现语言不限，比如 C/C++或 Python 或其它 GPL 均可。
- 核心代码结构示例见课件“第四章”：

步骤 4：利用 fact 程序进行测试与生成

- 准备测试源：编写或获取 Fact 源程序（通常是一个计算阶乘的简单程序，包含循环和算术运算，适合测试指令生成）。
- 生成中间代码：
  - 将 Fact 源程序编译为中间表示（IR）手工转换为 QTAC 代码，或者直接手动编写对应的 QTAC 程序 `fact.qtac`。
  - 示例迭代计算的中间代码：`LABEL l0; t1=2; IF n<t1 THEN l1 ELSE l2; LABEL l1; RETURN a; GOTO l3; LABEL l2; t2=n-1; t3=n*a; n=t2; a=t3; GOTO l0; LABEL l3;`
  - 示例递归计算的中间代码（略）。
- 运行生成器：
  - 执行步骤 3 编写的程序，输入`fact.qtac`，输出`fact_asm.s`。
  - 命令示例：`./my_codegen < fact.qtac > fact_asm.s`

步骤 5：在鲲鹏环境下汇编与运行

- 完善`fact_asm.s`
  - 由于不是完整的`main`函数入口，需要需编写 C 语言驱动文件调用它。
  - 或者手工补全为完整的汇编程序，其中 fact 的参数 n 和 a 都要初始化。要使用函数调用约定。
- 上传文件：将生成的`fact_asm.s`上传至鲲鹏服务器。
- 汇编： 使用 GCC 或 AS 将汇编代码转换为对象文件。
- ```bash # 使用 GCC 汇编（推荐，自动处理架构细节） gcc -c fact_asm.s -o fact_asm.o ```
- 链接：
- ```bash gcc fact_asm.o -o fact_run ```
- 运行与验证：
- ```bash ./fact_run ```
- 采用调试方式检查计算结果是否符合预期的阶乘值（因为没有包含输出指令）。

步骤 5：调试与优化

- 如果运行报错（如`Illegal instruction`），说明生成的 ARM 指令有误。
- 使用`objdump`查看二进制文件，确认生成的机器码：
- ```bash objdump -d fact_run ```
- 使用`gdb`进行动态调试：
- ```bash gdb ./fact_run (gdb) break main (gdb) run ```

五、预期结果

- DFA 设计文档：包含状态转移表（也可采用压缩方式）。
- 代码生成器：能够正确解析 IR 并输出 ARM 汇编的执行程序。
- 运行截图：在鲲鹏终端上成功编译并运行 Fact 程序，踪迹断点检查表明计算结果正确（如初始化时 n=5，a=0，运行结束前 a=120）。

六、常见问题与排查

问题现象	可能原因	解决方案
汇编报错	生成的指令助记符错误（如混用了 x86 指令）	检查 DFA 的指令模板库，确保输出的是 ARM64 语法（如`MOV`而非`movl`）。
段错误	寄存器使用冲突或栈指针未对齐	检查 DFA 的寄存器分配逻辑，确保 ARM64 的栈对齐要求（16 字节）。
链接错误	缺少`_start`或`main`符号	确保生成的汇编文件中包含入口点，或正确链接了 C 运行时库。

七、实验总结

通过本次实验，学生应能初步理解从中间语言到机器码的映射机制，掌握利用 DFA 进行指令选择的算法实现，并熟悉鲲鹏 ARM 架构的底层汇编与运行机制。